

CMPS 132: Final Project Report

Tic-Tac-Toe Game

By Samantha Hulick

Project Description:

Two-player Tic-Tac-Toe game that uses a 3x3 playing board. The game utilizes the terminal to alternate turns between the players, 'X' and 'O', keeping track of the wins for each player.

Approach and Logic Used:

I started my program by brainstorming all of the methods I would need to implement the game. Some of these methods include `print_board()`, `clear_board()`, `play()`, etc. Originally, I kept the implementation pretty basic with a board that is printed upon each play and each player taking turns filling spaces until one player wins or a draw occurs, then ending the program. However, I decided to deepen the playability of my game by allowing multiple games to be played, keeping score of the wins of each player in the variables `self.x_win_count` and `self.o_win_count`. Inside the main method `play()`, a while loop is implemented to continuously take input for plays, until the user decides to not play again (when `game_end` variable is set to `True`). I also knew that alternating players' turns was an important feature to add, therefore I decided to use the variable `self.turn_counter` that would increase by 1 every turn made. Therefore, if the turn counter was an even number (`self.turn_counter % 2 == 0`), it would be player X's turn, and if the turn counter was an odd number, it would be player O's turn. I decided that after each game, I would not reset the `self.turn_counter` variable back to 0, in order to allow the loser of the previous game to start the next game, as per Tic-Tac-Toe rules. Additionally, before each play is added to the board, the input of the row and column variables given by the player are validated using a try and except block. Therefore, if the player inputs an index that is outside of the domain (0-2), a coordinate that is already taken by a previous play, or any value that is not an integer, their turn will continue and they will be given a short sentence that explains why their input wasn't accepted. If it is accepted, the program will simply use indexing of `self.board` to update the play on the board. After the play is added to the board, the `check_winner()` function will check whether there are 3 of the same symbol in a row (either horizontally, vertically, or diagonally), using list indexing and a for loop of `range(3)` since there will always be 3 rows and 3 columns. If the `check_winner()` function returns `True`, the win count for that player is increased by 1 and the terminal will ask the user if they want to play another game through the function `replay()` that when `True`, clears the board through the `clear_board()` method and allows the user to play more games. When `replay()` is `False`, `play()` ends the while loop by setting `game_end` to `True` and prints a thank you message. Finally, at the bottom of the code, you'll notice the welcome message as well as the creation of the `TicTacToe` object that actually allows the game to be played in the terminal.

Data Structures Used:

In order to demonstrate knowledge of use and allow easier implementation in the future, I decided to create the class called TicTacToe. In this class, all of the methods needed to play the game, update the score, reset the board, etc. are included.

```
class TicTacToe():
> def __init__(self):~
> def print_board(self):~
> def play(self):~
> def check_winner(self, player):~
> def increase_win_count(self, player):~
> def is_draw(self):~
> def is_full(self):~
> def replay(self):~
> def clear_board(self):~
```

Additionally, for my playing board, I used a two-dimensional list to represent the 3x3 board that Tic-Tac-Toe is usually played on. The board consists of 3 rows that each start with 3 items of empty spaces (‘ ‘) that will be updated with plays as the game continues.

```
def __init__(self):
    self.board = [
        [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
        [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
        [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']]
```

Challenges Faced:

Some of the biggest challenges faced with the code was figuring out how to transition to each turn as well as implementing the ability to play multiple games. Originally, I used a recursive algorithm within play() that would finish each turn (or mark on the playing board) with a recursive call to play() for the next turn that ended up creating too many branches and difficulties in fixing bugs, especially with possible errors in the turn counter. Therefore, I settled on using a while loop in play() instead that relied on the variable game_end that could be turned to True when the player decides not to play another game. This while loop kept the implementation more predictable and easier to maintain.

Possible Improvements:

Currently, the aspect that makes the game most difficult to follow along is the row/column input implementation. Although having the player directly input the index of the row and column of their play makes it easier to index the two-dimensional playing board list, this could increase the difficulty of playing for the user because it requires them to count the row/column index before making a play. If I could improve the playability of the game, I would update it so that clicking an empty space would fill it with the symbol of the play like a classic Tic-Tac-Toe game.